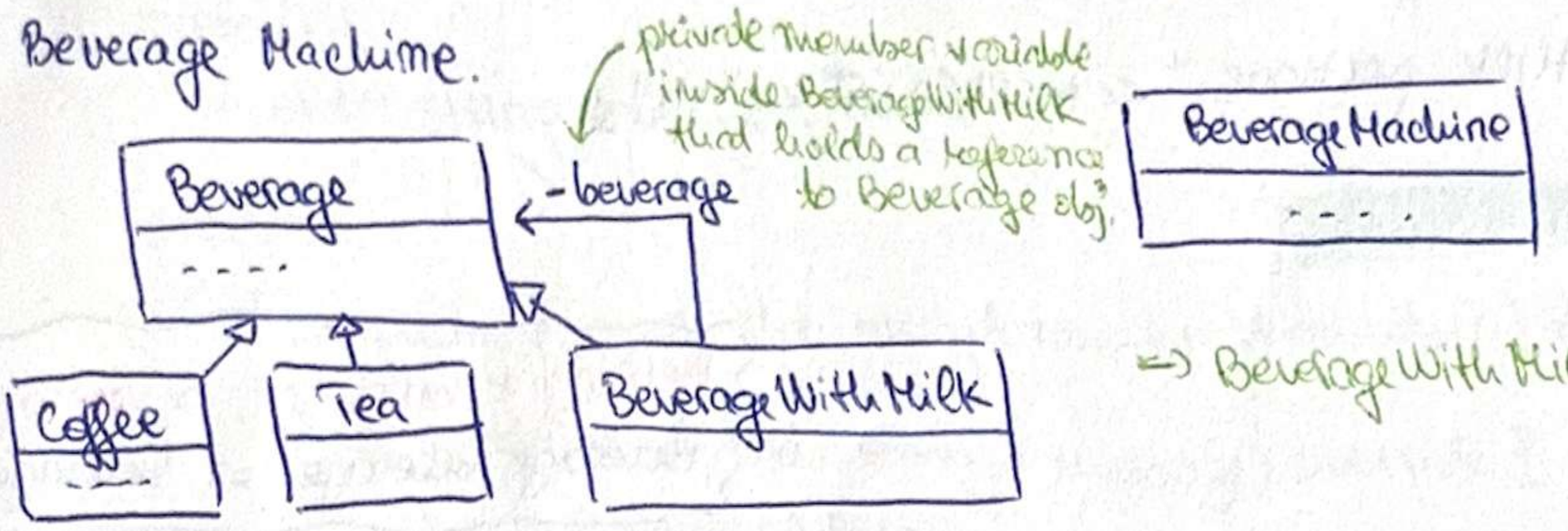
„Exam guide"-git!

3. Beverage Machine.



⇒ BeverageWithMilk IS-A Beverage
HAS-A Beverage

a) class Beverage {
protected:
std::string description; // Coffee/Tee may need access to it.
public:
Beverage(const std::string& description): description{description}{}
virtual double price() const = 0;
virtual void print() const {
cout << description << " " << price() << "\n";
}
virtual ~Beverage() = default;
};

b) class Coffee : public Beverage {
public:
Coffee(): Beverage{"Coffee"}{} // initialize the description by calling the base class constructor with the descriptions "Coffee", "Tea".
double price() const override {
return 2.5;
}
};

class Tea: public Beverage {
public:
Tea(): Beverage{"Tea"}{}
double price() const override {
return 1.5;
}
};

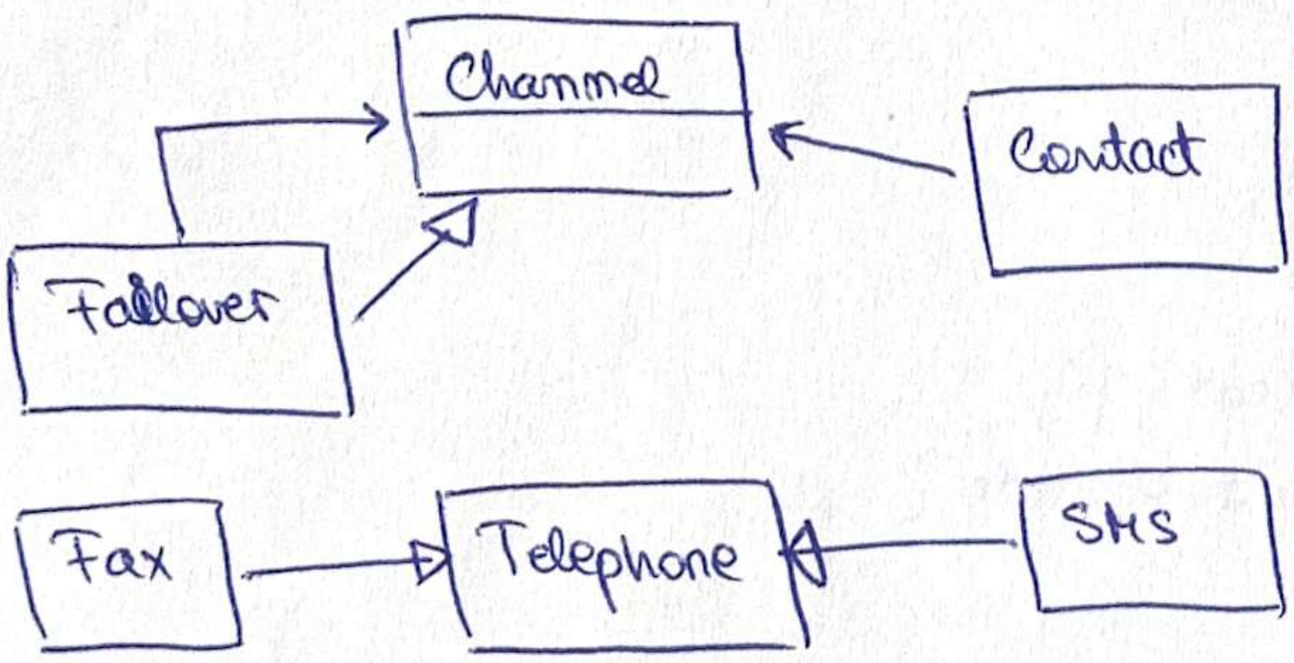
c) class BeverageWithMilk: public Beverage { (POLYMORPHISM)
private:
Beverage* beverage; // because it might be new Coffee() or new Tea() ...
int milkCount; => needs DESTRUCTOR!
public:
BeverageWithMilk(Beverage* beverage, int milkCount): Beverage{"with milk"},
beverage{beverage}, milkCount{milkCount}{};
double price() const override {
return beverage->price() + milkCount * 0.5;
}

```
void print() const override {
    beverage->print();
    std::cout << "Milk portions: " << milkCount << "\n";
}

~BeverageWithMilk() override {
    delete beverage;
}
```

Beverage* beverage => beverage.p
Beverage beverage => beverage.

d)
```
class BeverageMachine {
public:
    void prepare(const std::string& beverageType, int milkCount){
        Beverage* beverage;
        if (beverageType == "Tea")
            beverage = new Tea{};
        else
            beverage = new Coffee{};
        if (milkCount > 0) {
            beverage = new BeverageWithMilk(beverage, milkCount);
        beverage->print();
        std::cout << "Price: " << beverage->price() << "\n";
        delete beverage;
    }
```

u row 2^4 (Adder) (912)



a)
```
class Channel {
public:
    virtual void send(const std::string& message) = 0;
    virtual ~Channel() = default;
};
```

c) class Telephone: public Channel {

```
protected:
    std::string number;
public:
    Telephone(const std::string& number): number{number}{};
    void send(const std::string& message) override {
        int r = rand()%10;
        if(r==0)
            throw std::runtime_error("Line busy");
        std::cout<< "dialing" <<number<<"\n";
    }
};
```

e) class Fax: public Telephone {

```
public:
    Fax(const std::string& number): Telephone{number}{};
    void send(...) override{
        Telephone::send(message);
        std::cout<< "sending fax..." <<message<<"\n";
    }
};
```

class SMS: public Telephone{

```
public:
    SMS(...): Telephone{number}{};
    void send(...) override{
        Telephone::send(message);
        std::cout<<"sending SMS..." <<message<<"\n";
    }
};
```

d) class Failover: public Channel{

```
private:
    Channel* first;
    Channel* second;
public:
    Failover(Channel* first, Channel* second): first{first}, second{second}{}
    void send(....) override{
        try{
            first->send(message);
        }
        catch(std::exception&){
            second->send(message);
        }
    }
    ~Failover() override{ delete first; delete second;}
};
```

e) class Contact {

?

private:

std::vector<Channel*> channels;

public:

f) void addChannel(Channel* channel) {
channels.push_back(channel);
}

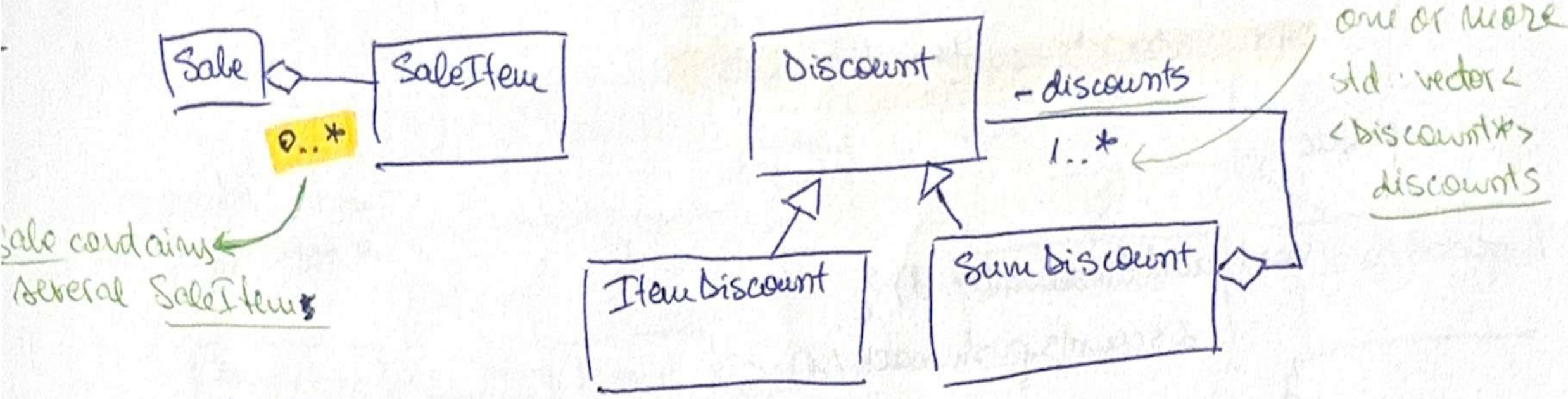
void sendAlarm(const std::string& message) {
for (auto channel: channels) {
try {
channel→send(message);
return;
}
catch (std::exception&) {
}
}
}

~Contact() {
for (auto channel : channels)
delete channel;
}
};

f) Contact* createContact(const std::string& number) {
?
auto* contact = new Contact{};
contact→addChannel(new Failover{new Fax{number}, new SMS{number}});
~~contact→addChannel(new Failover{new Fax{number}, new SMS~~
contact→addChannel(new Telephone{number}, new Failover{new Fax{number},
new SMS{number}});
return contact;
}

int main() {
Srand(time(NULL));
Contact* contact = createContact("0759110078");
contact→sendAlarm("alarm message");
delete contact;
return 0;
}

u 2018 - 1ª (June) (row 1 = row 2)



a) class SaleItem {
private:
int code;
double price;
public:
SaleItem(int code, double price): code{code}, price{price} { }{
int getCode() const { return code; }.
double getPrice() const { return price; }
};

class Sale {
private:
std::vector<SaleItem*> items;
public:
void addItem(const SaleItem& item) {
items.push_back(item);
}
const std::vector<SaleItem*>& getItems() const {
return items;
}
};

b) class Discount {
public:
virtual double calc(const Sale& s) const = 0;
virtual ~Discount() = default;
};

c) class ItemDiscount : public Discount {
private:
int code, percentage;
public:
ItemDiscount(int code, int percentage): code{code}, percentage{percentage}
double calc(const Sale& s) const override{ { }
double result = 0;
for(const auto& item : s.getItems()) {
if(item.getCode() == code) {
result += item.getPrice() * percentage / 100.0;
}
}
return result;
};

d) class SumDiscount : public Discount {

private:

std::vector<Discount*> discounts;

public:

void add(Discount* d) {

discounts.push_back(d);

}

double calc(const Sale& s) const override {

double total = 0;

for (auto d: discounts) {

total += d->calc(s);

}

return total;

}

~SumDiscount() override {

for (auto d: discounts)

delete d;

}

e) int main() {

Sale sale;

sale.addItem(SaleItem{1,100});

sale.addItem(SaleItem{2,200});

sale.addItem(SaleItem{3,300});

SumDiscount discount;

discount.add(new ItemDiscount{1,10}); 10%

discount.add(new ItemDiscount{2,15}); 15%

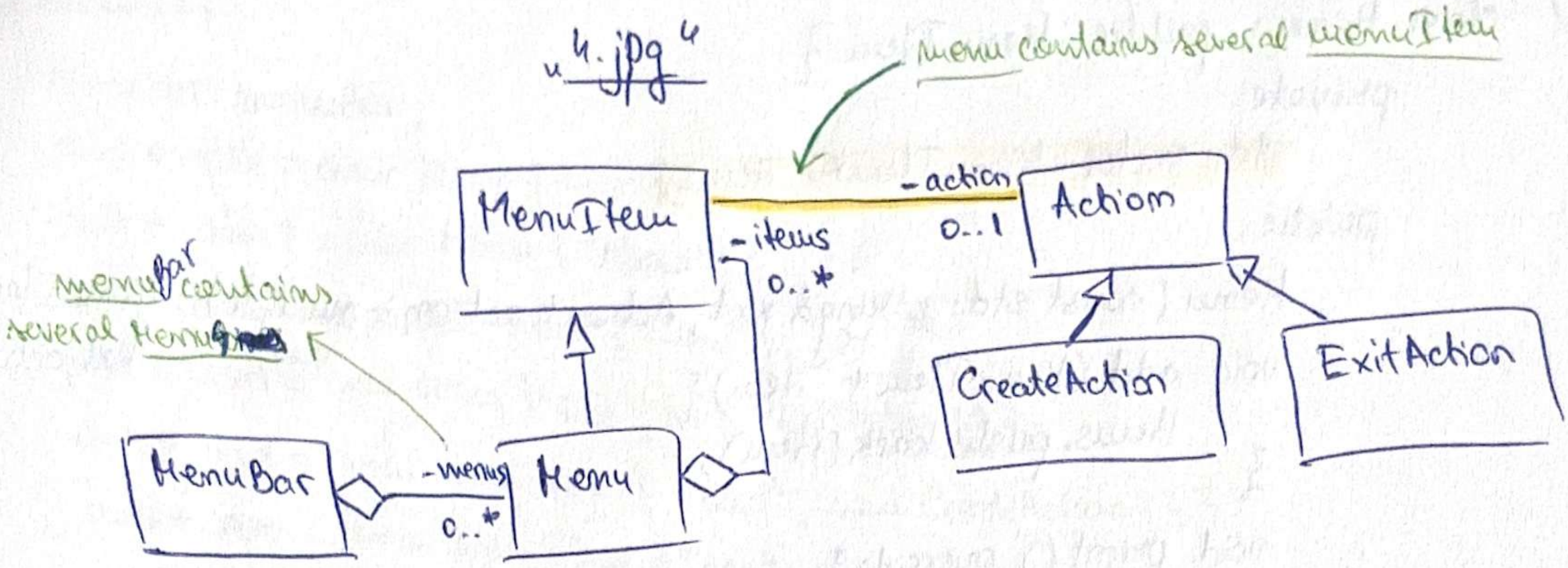
std::cout << discount.calc(sale);

}

f) memory correctly managed because the only dyn. allocations are stored inside vector<Discount*> => destructor called automatically when function main() goes out of scope.

! BUT when we do | SumDiscount* d = new SumDiscount{}; |, destructor is NOT automatically called. => | delete d | needed

3.



a) class Action{
public:
virtual void execute() = 0;
virtual ~Action() = default;
};

class CreateAction : public Action {
public:
void execute() override {
std::cout << "Create file\n";
}
};
class ExitAction : public Action {
public:
void execute() override {
std::cout << "Exit application\n";
}
};

b) class MenuItem {
protected:
std::string text;
Action* action;
public:
MenuItem(const std::string& text, Action* action=nullptr): text{text},
action{action}{}
virtual void print(){
std::cout << text << "\n";
}
virtual void clicked() {
std::cout << text << "\n";
if(action != nullptr)
action->execute();
}
virtual ~MenuItem(){
delete action;
}
};

0..1 — in case it's 0

c) class Menu: public MenuItem {

```
private:
    std::vector<MenuItem*> items;
public:
    Menu(const std::string& text, Action* action = nullptr) : MenuItem{
                                                    text, action}{}
    void add(MenuItem* item){
        items.push_back(item);
    }
    void print() override{
        std::cout<<text<<"\n";
        for(auto item: items)
            item->print();
    }
    MenuItem* getItem(int index){
        return items[index];
    }
    ~Menu() override{
        for(auto item: items)
            delete item;
    }
};
```

d) class MenuBar{

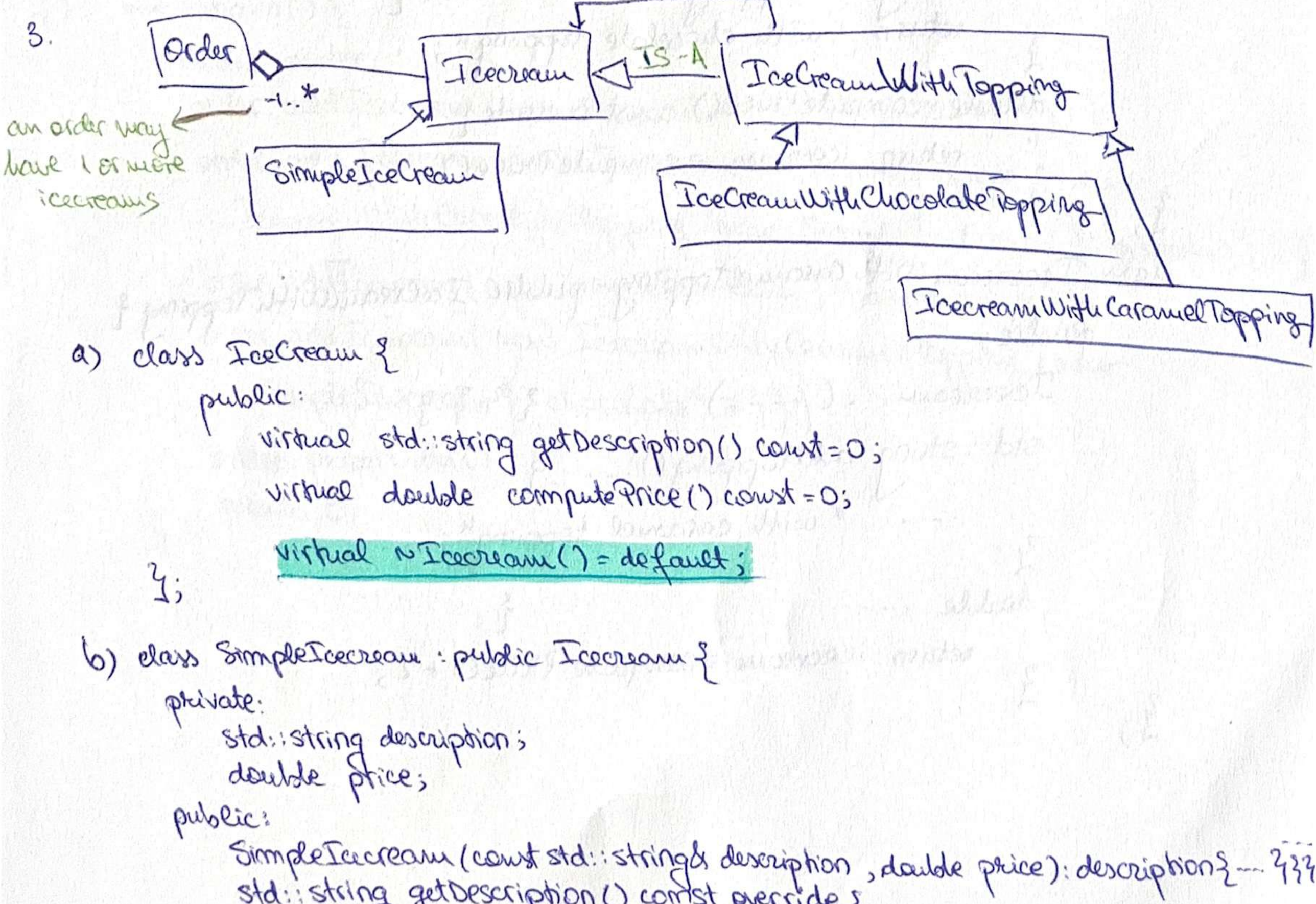
```
private:
    std::vector<Menu*> menus;
public:
    void add(Menu* menu){
        menus.push_back(menu);
    }
    void print(){
        for(auto menu: menus)
            menu->print();
    }
    Menu* getMenu(int index){
        return menus[index];
    }
    ~MenuBar(){
        for(auto menu: menus)
            delete menu;
    }
};
```

```
int main(){
    MenuBar menuBar;
    auto* file = new Menu{"File"};
    auto* about = new Menu{"About"};
    auto* newMenu = new Menu{"New"}, ← File
    auto* exitMenu = new Menu{"Exit", new ExitAction{}};
    auto* text = new MenuItem{"Text", new CreateAction{}};
    auto* cpp = new MenuItem{"C++", new CreateAction{}};
    newMenu->add(text);
    newMenu->add(cpp);
    file->add(newMenu);
    file->add(exitMenu);
    menuBar.add(file);
    menuBar.add(about);
    menuBar.print();
    newMenu->getItem(1)->clicked(); // file->new->c++
    exitMenu->clicked(); //exit
    return 0;
}
```

MenuItem

Exam 2026 (915)

3.



a) class IceCream {
    public:
        virtual std::string getDescription() const=0;
        virtual double computePrice() const=0;

        virtual ~Icecream() = default;
};

b) class SimpleIcecream : public Icecream {
    private:
        std::string description;
        double price;
    public:
        SimpleIcecream(const std::string& description, double price): description{...}{}
        std::string getDescription() const override {
            return "simple ice cream with" + description;
        }
}

```
    double computePrice() const override {
        return price;
    }
};
```

c) class IcecreamWithTopping : public Icecream{
protected:
    Icecream* icecream;

#1 one-to-one

You have no constr for Icecream, so it is OK! ✓

public:
    IcecreamWithTopping(Icecream* icecream): icecream{icecream}{}
    virtual std::string addTopping() const = 0;
    std::string getDescription() const override {
        return icecream->getDescription()+" "+addTopping();
    }
    ~IcecreamWithTopping() override {
        delete icecream;
    }
};

d) class IcecreamWithChocolateTopping: public IcecreamWithTopping {
public:
    Icecream.... (Icecream* icecream): IcecreamWithTopping{icecream}{}
    ⇒ It has a constructor ✓
    std::string addTopping() const override {
        return "with chocolate topping";
    }
    double computePrice() const override{
        return icecream->computePrice() + 3;
    }
};

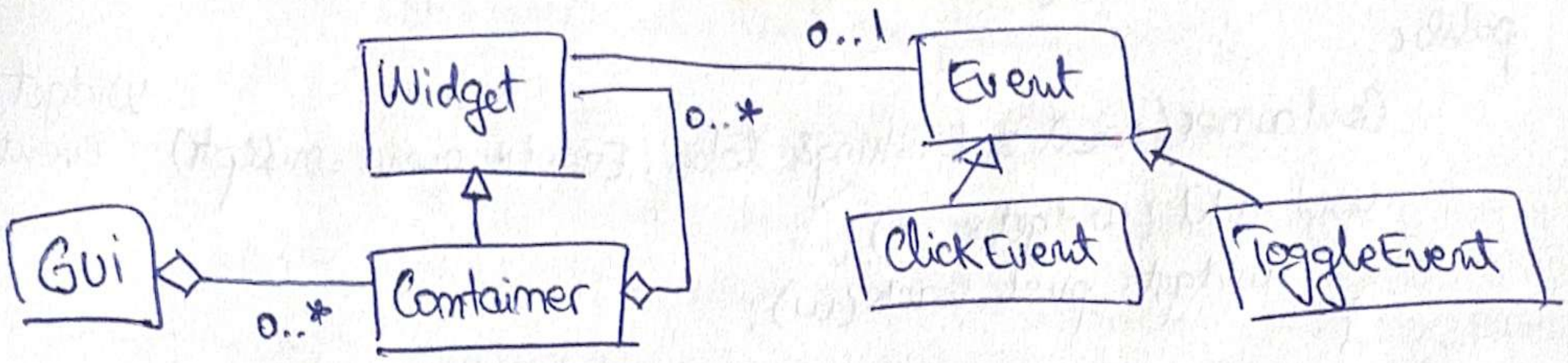
class IcecreamWithCaramelTopping: public IceCreamWithTopping {
public:
    Icecream...(----): ..... {{{}}
    std::string addTopping()---{
        --- "with caramel topping";
    }
    double ---   {
        return icecream->computePrice() + 2;
    }
};

```
class Order {
    private:
        std::vector<Icecream*> icecreams;

    public:
        void addIcecream(Icecream* icecream) {
            icecreams.push_back(icecream);
        }
        void printOrder() {
            std::sort(icecreams.begin(), icecreams.end(), [](Icecream* a,
                Icecream* b) {
                    return a->computePrice() > b->computePrice();
                });
            for(auto icecream: icecreams) {
                std::cout << icecream->getDescription() << " "
                          << icecream->computePrice() << "\n";
            }
        }
        ~Order() {
            for(auto icecream: icecreams)
                delete icecream;
        }
};

7) int main() {
    Order order;
    order.addIcecream(new SimpleIceCream{"vanilla", 5});
    order.addIcecream(new IcecreamWithCaramelTopping(new
        IcecreamWithChocolateTopping(new SimpleIceCream{"pistachio", 6})));
    order.addIcecream(new SimpleIcecream{"hazelnuts", 8});
    order.addIcecream(new IcecreamWithCaramelTopping(new
        SimpleIcecream{"chocolate", 7}));
    order.printOrder();
    return 0;
}
```

1..*

Exam 2026 (916)

3. (Same as Menus)



a)
```
class Event {
    public:
        virtual void trigger() = 0;
        virtual ~Event() = default;
};
class ClickEvent : public Event {
    public:
        void trigger() override {
            cout << "Clicked event triggered";
        }
};
class ToggleEvent : public Event {
    public:
        void trigger() override {
            cout << "Toggle event triggered";
        }
};
```

b)
```
class Widget {
    protected:
        std::string label;
        Event* event;
    public:
        Widget(const std::string& label, Event* event = nullptr) : label{label},
                                                                   event{event} {}
        virtual void render() {
            std::cout << label << "\n";
        }
        virtual void interact() {
            std::cout << label << "\n";
            if(event != nullptr) {
                event->trigger();
            }
        }
        virtual ~Widget() {
            delete event;
        }
};
```

0..1

c) class Container: public Widget {

0..*

```
    private:
        std::vector<Widget*> widgets;
    public:
        Container(const std::string& label, Event* event=nullptr): Widget{label, event} { }
        void add(Widget* w){
            widgets.push_back(w);
        }
        void render() override {
            std::cout << label << "\n";
            for(auto widget:widgets)
                widget->render();
        }
        Widget* getWidget(int index) {
            return widgets[index];
        }
        ~Container() override {
            for(auto widget: widgets){
                delete widget;
            }
        }
};
```

d) class Gui {

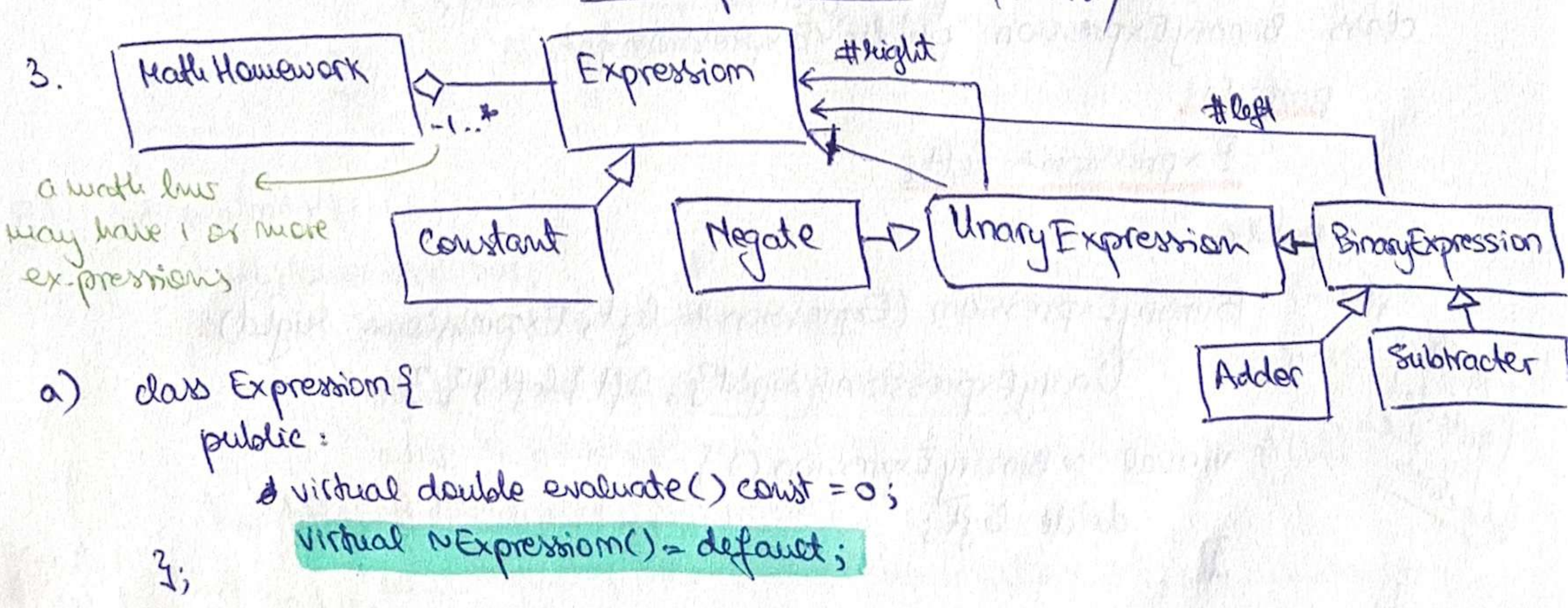
0..*

```
    private:
        std::vector<Container*> containers;
    public:
        void add(Container* c){
            containers.push_back(c);
        }
        void render() {
            for(auto c: containers)
                c->render();
        }
        Container* getContainer(int index){
            return containers[index];
        }
        ~GUI(){
            for(auto c: containers){
                delete c;
            }
        }
};
```

```
int main() {
    GUI gui;
    auto* MainPanel = new Container {"MainPanel"};
    auto* HelpPanel = new Container {"HelpPanel"};
    auto* SettingsPanel = new Container {"SettingsPanel"};
    auto* ExitButton = new Widget {"ExitButton", new ToggleEvent {}};
    auto* CppButton = new Widget {"CppButton", new ClickEvent {}};
auto* TextButton = new Widget {"TextButton", new ClickEvent {}};
SettingsPanel -> add(TextButton);
 SettingsPanel -> add (CppButton);
 MainPanel -> add (SettingsPanel);
 MainPanel -> add (ExitButton);
 gui.add (MainPanel);
 gui.add (HelpPanel);
CppButton -> interact(); // main panel -> settings panel -> cpp button.
ExitButton -> interact(); //exit button
return 0;
}
```

"exam oop written" (weird)

3.



a) class Expression {
public:
virtual double evaluate() const = 0;
virtual ~Expression() = default;
};

b) class Constant : public Expression {
private:
double value;
public:
Constant (double value) : value{value} { }
double evaluate() const override {
return value;
}
};

c) UnaryExpression aggregates an expression; Negate also aggregates an expre

```
class UnaryExpression : public Expression {
    protected:
        Expression* right;          // → Negate will need access. (class)

    public:
        UnaryExpression (Expression* right) : right{right} {}

        double evaluate() const override {
            return right->evaluate();
        }
        virtual ~UnaryExpression() {
            delete right;
        }
};
```

which one??

```
class Negate : public UnaryExpression {
    public:
        Negate (Expression* expr) : UnaryExpression {expr} {}
        double evaluate() const override {
            return -expr->evaluate();
        }
};
```

d) BinaryExpression aggregates two other expressions.

```
class BinaryExpression : public UnaryExpression {
    protected:
        Expression* left;
    public:
        BinaryExpression (Expression* left, Expression* right):
            UnaryExpression{right}, left{left} {}
        virtual ~BinaryExpression () {
            delete left;
        }
};
```

right is protected in the base class

... and we inherit from it!

e) class Adder: public BinaryExpression {

not added as fields because BinaryExpression has itself those in the constructor!

```
    public:
        Adder (Expression* left, Expression* right) : BinaryExpression {left, right}
        double evaluate() const override {
            return left->evaluate() + right->evaluate();
        }
};
```

lass Subtracter: public BinaryExpression{
public:
Subtracter(Expression* left, Expression* right): BinaryExpression{left, right}}
double evaluate() const override {
return left->evaluate() - right->evaluate();
};

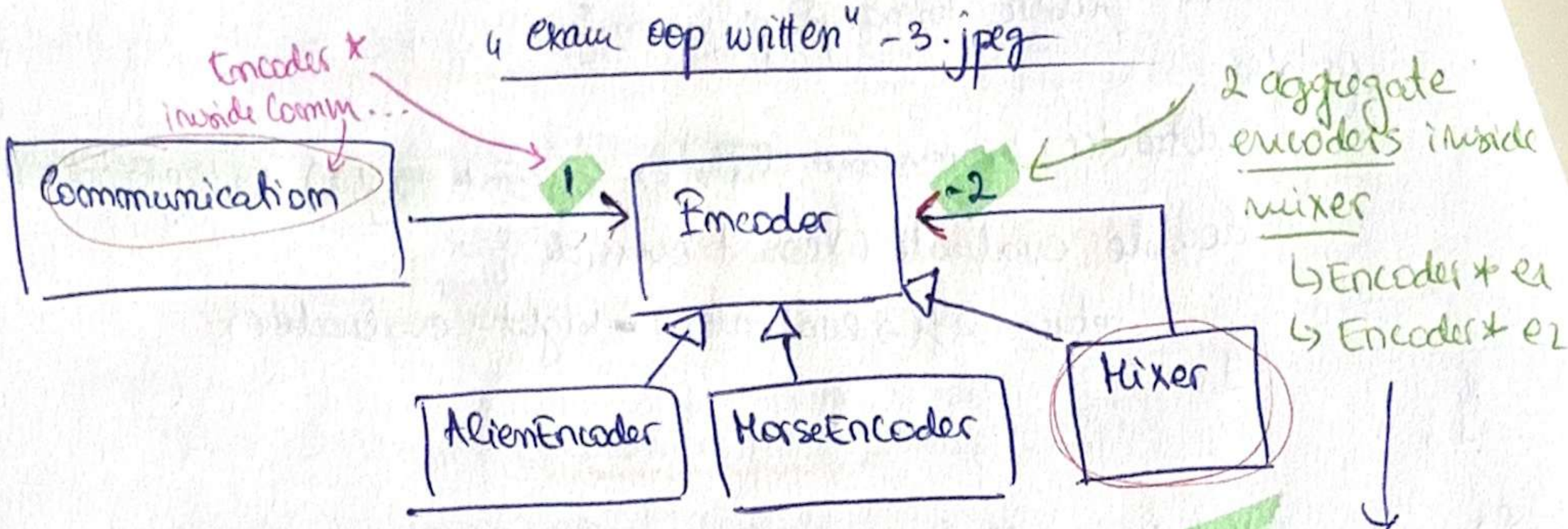
f) class MathHomework {
private:
std::vector<Expression*> expressions;
public:
void addExpression(Expression* e){
expression.push_back(e);
}
std::vector<double> getResults() const {
std::vector<double> results;
for(auto e: expressions)
results.push_back(e->evaluate());
return results;
}
~MathHomework(){
for(auto e: expressions)
delete e;
}
};

g) int main(){
MathHomework hw;
hw.addExpression(new Adder(~~new~~
new Negate(new Constant(5)),
new Subtracter(new Constant(9), new Constant(3))));

-5+(9-3)
-(4+2)-(-10) =
=-((4+2)-10)

hw.addExpression(new Subtracter(
new Negate(new Adder(new Constant(4), new Constant(2)),
new Negate(new ~~Negate~~ Constant(10))));
for(double result: hw.getResults())
cout<<result<<" ";
return 0;
}

3.



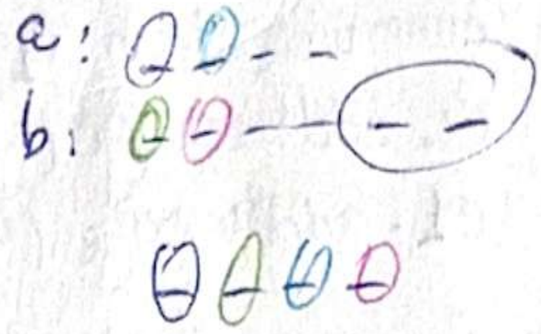
a) class Encoder {
public:
virtual std::string encode(const std::string& msg) = 0;
virtual ~Encoder() = default;
};

b) class AlienEncoder : public Encoder {
private:
std::string header;
public:
AlienEncoder(const std::string& header) : header{header} { }
std::string encode(const std::string& msg) override {
return header + msg;
}
};

c) class HorseEncoder : public Encoder {
public:
std::string encode(...) override {
std::string result;
for(char c : msg)
result += "...";
return result;
}
};

d) class Mixer : public Encoder {
private:
Encoder* e1;
Encoder* e2;
public:
Mixer(Encoder* e1, Encoder* e2) : e1{e1}, e2{e2} { }
std::string encode(...) override {
std::string a = e1 → encode(msg);
std::string b = e2 → encode(msg);
std::string result;

```
int m = std::min(a.size(), b.size());
for(int i=0; i<m; i++){
    result += a[i];
    result += b[i];
}
if(a.size() > b.size())
    result += a.substr(m);
if(b.size() > a.size())
    result += b.substr(m);
    return result;
}
~Mixed() override {
    delete e1;
    delete e2;
}
};
```



e)

```
class Communication{
    private:
        std::vector<std::string> messages;
        Encoder* encoder;
    public:
        Communication(Encoder* encoder): encoder{encoder}{}
        void addMessage(const std::string& msg){
            messages.push_back(msg);
        }
        std::string communicate(){
            std::sort(messages.begin(), messages.end());
            std::string result;
            for(const auto& msg: messages){
                result += encoder->encode(msg);
                result += '\n';
            }
            return result;
        }
        ~Communication(){
            delete encoder;
        }
};
```

by VALUE => copy of each element!

when auto msg vs const auto& msg

NO COPY, NO modify the original

f) int main() {

Communication c1{new HorseEncoder{}};
c1.addMessage("hello"); } ↳ each comm. obj. contains 2 messages
c1.addMessage("abc");

Communication c2{new Mixer{new AlienEncoder{"ET"},
new HorseEncoder{}}};

c2.addMessage("hellloooo");
c2.addMessage("aacbbcc");

std::cout << c1.communicate();
Std:: cout << c2.communicate();
return 0;

}

2026 Exam guide

! 3. a) class Checker {
(Harder) public:
virtual bool check(Section* s) = 0;
virtual ~Checker() = default;
};

b) class TitleChecker : public Checker {
public:
bool check(const Section* s) override {
return s→getTitle().length() > 2;
}
};

class ContentChecker : public Checker {
public:
bool check(const Section* s) override {
~~std::string content = s→getContent();~~
~~return std::isupper(static_cast~~
return !s→getContent().empty() &&
std::isupper(s→getContent()[0]);
}
};

c) class DoubleChecker : public Checker {
private:
Checker* c1;
Checker* c2;
public:
DoubleChecker(Checker* c1, Checker* c2) : c1{c1}, c2{c2} {}
bool check(Section* s) override {
return c1→check(s) && c2→check(s);
}

~DoubleChecker() override {
    delete c1;
    delete c2;
}
};

~~class Preface : public Section{~~
class Section {
protected:
    std::string title;
    std::string content;
    Checker* checker;
public:
    Section(std::string title, std::string content, Checker* checker):
        title{title}, content{content}, checker{checker}{}
    std::string getTitle()...
    std::string getContent()...
    virtual void addSection(Section* section) = 0;
    ~~virtual~~ void generate(){
        if (checker->check(this)){
            std::cout << title << "\n";
            std::cout << content << "\n";
        }
    }
    virtual ~Section(){
        delete checker;
    }
};

e) class Preface : public Section{
public:
    Preface(std::string title, std::string content, Checker* checker):
        : Section{title, content, checker}{}
    void addSection(Section* s) override{
        throw std::runtime_error("Preface cannot have sections");
    }
};

f) class Chapter : public Section{
    ~~public~~: private:
        ~~Preface~~ std::vector<Section*> sections;
public:
    Chapter(std::string title, std::string content, Checker* checker):
    : Section{title, content, checker}{}
    void addSection(Section* s) override{
        sections.push_back(s);
    }
};

```
void generate() override {
    if(checker->check(this)){
        std::cout<< title <<"\n";
        std::cout<< content <<"\n";
        for (auto s: sections)
            s-> generate();
    }
}

~Chapter() override{
    for (auto s: sections)
        delete s;
}
};
```

```
int main(){
    Section* preface = new Preface{"Preface", "this is...", new ContentChecker{}};
    Section* chapter = new Chapter{"Chapter1", "content", new TitleChecker{}};
    chapter->addSection(new Chapter{"Subchapter1", "valid content",
        new DoubleChecker{new TitleChecker{}, new ContentChecker{}}});
    preface->generate();
    chapter->addSection(new Chapter{"No", "invalid content.",
        new DoubleChecker{new TitleChecker{}, new ContentChecker{}}});
    preface-> generate();
    chapter-> generate();
    delete preface;
    delete chapter;
    return 0;
}
```